

Comenzando con Docker

Sumario

Comenzando con Docker.....	1
Introducción.....	2
Ejecutar un contenedor docker.....	3
Crear nuestra primera imagen.....	3
Creando una imagen con un servicio web.....	5
Creando una imagen docker para react y desplegarla en otro ordenador.....	6
Creando una imagen docker para laravel.....	9
Imagen docker para laravel con persistencia en base de datos.....	14

Introducción

Según wikipedia: “Docker es un proyecto de código abierto que automatiza el despliegue de aplicaciones dentro de contenedores de software, proporcionando una capa adicional de abstracción y automatización de virtualización de aplicaciones en múltiples sistemas operativos. Docker utiliza características de aislamiento de recursos del kernel Linux, tales como cgroups y espacios de nombres (namespaces) para permitir que "contenedores" independientes se ejecuten dentro de una sola instancia de Linux, evitando la sobrecarga de iniciar y mantener máquinas virtuales”

La parte realmente importante de Docker es que **permite tener en una sandbox las dependencias de software para desplegar una aplicación que hayamos desarrollado** (al estilo de lo que haría una máquina virtual pura donde hayamos instalado nuestra app) y llevándonos ese contenedor a otra máquina todo funcionará igual (si tiene docker instalado), pero de una forma más eficiente que si fuera una máquina virtual pura

Docker, a diferencia de una máquina virtual: virtualbox, vmware,... se ejecuta **directamente sobre el nucleo (kernel) de la máquina anfitriona siendo así más eficiente que una virtualización pura**. Como contra implica que únicamente se puede ejecutar en máquinas linux de esa forma. Las máquinas windows al instalar docker lo que hacen es “emular” el comportamiento de docker usando una máquina virtual. Esto es, no es tan eficiente como en linux

Bien, imaginemos que tenemos una .ova de una máquina virtual, eso es una especie de imagen estática, donde tendremos nuestra aplicación en un estado funcional que deseemos.

Sabemos que esa .ova tenemos que importarla para poder ejecutarla y en los ordenadores donde importemos la .ova tendremos exactamente la misma máquina virtual instalada. Supongamos que la hemos importando en el PC1 y en el PC2 A partir de ese momento, las variaciones de la máquina virtual del PC1 que se hagan lo harán diferente de la máquina virtual en el PC2

Lo anterior tiene así la semejanza con Docker en su concepto de: imágenes y contenedores. Así nosotros creamos una imagen docker (el equivalente de la .ova) y luego en el ordenador que se quiera ejecutar se crear un contenedor tomado de esa imagen (el equivalente a la máquina virtual que has importado de la .ova)

Ejecutar un contenedor docker

Hay montones de imágenes docker disponibles. Por ejemplo hay imágenes base de ubuntu. Probemos el siguiente comando:

```
docker run --rm --name pruebas -it ubuntu /bin/bash
```

Al ejecutar lo anterior se descarga una imagen de ubuntu (el equivalente de la ova), luego crea un contenedor basado en esa imagen llamado: pruebas y te abre una shell bash (eso es por la opción: -it /bin/bash) Cuando cerremos la shell el contenedor docker se borrará automáticamente (eso es por la opción: --rm)

Crear nuestra primera imagen

Podemos empezar creando una carpeta donde pondremos un fichero index.php con el contenido:

```
<?php
    echo "Hola mundo! Esto es una app en php ejecutando desde docker \n\n";
?>
```

Luego otro fichero llamado Dockerfile con el contenido:

```
# Usa la imagen oficial de PHP
FROM php:8.2-cli

# Copia el archivo PHP al contenedor
COPY index.php .

# Define el comando por defecto para ejecutar el script PHP
CMD ["php", "index.php"]
```

Ahora en una terminal dentro de la carpeta:

```
docker build -t imagenphpcli .
```

Al hacerlo ejecutará el proceso de creación de la imagen con el nombre: holamundophpcli basándose en la información almacenada en el fichero Dockerfile (el punto que ponemos en el comando es para indicarle la ubicación del fichero Dockerfile)

Podemos ver la imagen creada ejecutando:

```
docker images
```

Y podemos lanzar la aplicación usando como antes docker run:

```
docker run --rm --name pruebas imagenphpcli
```

Ok, con lo anterior hemos creado y ejecutado nuestra primera app en docker. El proceso siempre es crear un Dockerfile en la raíz del proyecto y poner las instrucciones allí. Vamos ahora a entender que ha hecho el anterior Dockerfile:

1. FROM php:8.2-cli

La línea anterior le dice a docker que se base en una imagen ya creada (podría haber sido el ubuntu que usamos antes perfectamente). En este caso se descargará y preparará la imagen base para ejecutar php en su versión 8.2 (para consola, por eso: cli)

2. COPY index.php .

La línea anterior usa COPY. La instrucción COPY permite copiar desde el ordenador anfitrión al interior de nuestra imagen docker. El primer parámetro es el fichero/carpeta del ordenador anfitrión y el segundo parámetro la ubicación dentro de la imagen docker donde se copiará. En este caso se está copiando el fichero index.php que previamente hemos creado al interior del contenedor docker

3. CMD ["php", "index.php"]

Finalmente en la línea anterior le decimos el comando a ejecutar. Esto es, que php ejecute el fichero que le hemos pasado. Aquí puntualizar que también se usa ENTRYPOINT en lugar de CMD. Cuando se hacen imagenes para producción ENTRYPOINT puede ser más apropiado porque no permite cambios en lo que se va a ejecutar por defecto. Pero por ahora nos es indiferente

Creando una imagen con un servicio web

los contenedores docker habitualmente son para alojar servicios. Vamos a hacer una variación del Dockerfile anterior para que en lugar de en consola nos muestre la página en un servicio web:

```
# Usa la imagen oficial de PHP con Apache
```

```
FROM php:8.2-apache
```

```
# Copia el archivo PHP al directorio raíz del servidor web
```

```
COPY index.php /var/www/html/
```

```
# Documentamos para los lectores del Dockerfile en que puerto se lanza la app
```

```
EXPOSE 80
```

Los cambios están en la imagen base: `FROM php:8.2-apache` que ahora es el servidor web apache con php ya instalado. En el comando COPY ahora tenemos que copiar a la posición del contenedor donde se sirven las páginas web: `/var/www/html` y finalmente el otro comando: `EXPOSE 80` que tiene propósito documental/informativo. Simplemente ayuda a saber al lector del documento Dockerfile en que puerto está lanzado el servicio dentro del contenedor docker

podemos crear la imagen ahora con:

```
docker build -t imagenphpweb .
```

Y la lanzamos con:

```
docker run -p 8080:80 --rm --name pruebas imagenphpweb
```

La parte nueva es que aparece la opción: `-p` que hace mapeo de puertos. En este caso hace que el puerto dentro del contenedor docker 80 sea accesible desde la máquina anfitriona en el puerto 8080. Así que podemos ejecutar la app desde el navegador en el puerto 8080: <http://localhost:8080>

Creando una imagen docker para react y desplegarla en otro ordenador

1. Lo primero que tenemos que tener en cuenta es que necesitamos un fichero: Dockerfile (lo ubicamos en la raíz de nuestra carpeta de proyecto react) que es donde se detallan las instrucciones para crear nuestra imagen docker

Veamos un Dockerfile que permite desplegar una aplicación react:

```
FROM node:lts-alpine3.10 AS node-for-build

RUN mkdir app

WORKDIR /app

COPY . .

RUN npm install && npm run build

FROM nginx:stable-alpine3.20 AS nginx-server

COPY --from=node-for-build /app/build /usr/share/nginx/html

EXPOSE 80

CMD ["nginx","-g","daemon off;"]
```

La primera línea toma una imagen preexistente de node pensada para docker (es imagen oficial) que es LTS, en concreto la versión 3.10.

Esto es muy habitual, utilizar una imagen docker que ha desarrollado los encargados del proyecto Node y que ya venga con lo básico para que funcione node. De igual forma, sabemos que para una app java tendríamos que tener un JRE pues entonces podemos comenzar a partir de una imagen openjdk:17-alpine

La segunda línea: mkdir app crear una carpeta (recordar que nuestra imagen ahora mismo está vacía salvo lo que hayamos tomado de node) para que alojemos nuestra app la hemos llamado: app

La tercera línea le dice que las siguientes instrucciones las ejecutaremos en ese directorio: `WORKDIR /app`

La cuarta línea puede parecer confusa al principio: `COPY . .` ya que pareciera que estás copiando todo en el mismo sitio. Sin embargo, el comando `COPY` toma como referencia la carpeta de nuestro proyecto donde está el fichero `Dockerfile` como el primer punto, Siendo el segundo parámetro el destino (el segundo punto) , que siempre es una ubicación dentro de la imagen que estamos creando. Así la instrucción anterior copia todo el proyecto (salvo lo que esté puesto detallado en el fichero `.dockerignore`) de nuestra carpeta del proyecto a dentro de nuestro contenedor docker. En concreto en la carpeta `/app` ya que antes hemos dicho que es nuestra zona de trabajo (`workdir`)

La quinta línea: `RUN npm install && npm run build` es lo habitual cuando queremos crear una aplicación node. Sabemos que el `npm install` nos genera las dependencias (`node_modules` etc) de nuestro proyecto y luego ejecutamos un `build` que nos deja la app optimizada para el despliegue

La sexta línea: `FROM nginx:stable-alpine3.20 AS nginx-server` nos trae a nuestro contenedor la imagen del servidor web: `nginx` (en su versión estable 3.20) Así que podremos ejecutar aparte de node (que ya lo hemos hecho en las líneas anteriores) un servidor de páginas web `nginx`

La séptima línea: `COPY --from=node-for-build /app/build /usr/share/nginx/html` es un poco más compleja. Para entenderla tenemos que saber que Docker se construye con capas fijas. Esto es, cuando hicimos la primera línea nos creamos una capa que se traía todo el software de node. En la sexta línea nos hemos traído información de `nginx` que ha creado una capa nueva. Para acceder a la información que teníamos en el estado precedente usamos la etiqueta que generamos al principio: `node-for-build` para indicarle que queremos que nos traiga a esta nueva capa la carpeta que node nos generó al ejecutar el `build` y que lo ponga en el directorio que `nginx` usa para servir las páginas web

La octava línea puede sobrar: `EXPOSE 80` que le indica que quiere que la app esté disponible en el puerto 80 (que es el que tiene por defecto `nginx` y por eso podemos evitar poner esta línea)

Finalmente en la novena línea: `CMD ["nginx","-g","daemon off;"]` lanzamos `ngix` y queda por tanto, sirviendo nuestra app

2. Ejecutamos el comando para que nos cree la imagen docker usando el `Dockerfile` anterior (lo lanzamos en la carpeta raíz del proyecto)

```
docker build -t elnombredenuestraimagendocker .
```

La instrucción anterior, nos genera una nueva imagen docker usando el `Dockerfile` que hemos creado (el punto hace referencia a la ubicación donde está el `Dockerfile`)

Podemos ver la imagen creada listando las imagenes que tiene en este momento docker instaladas:

```
docker images
```

Como no le hemos puesto un número de versión nos habrá puesto un TAG: latest

Si hubiéramos querido un número de versión se lo ponemos mediante dos puntos: “:” al crear la imagen: `docker build -t elnombredenuestraimagendocker:1.0` .

3. Llevarnos la imagen al sitio que queremos. Para ello podemos hacerlo independientemente de internet

En el ordenador donde hemos creado la imagen:

```
docker save elnombredenuestraimagendocker > imagen.tar
```

En el ordenador donde queremos usar la imagen:

```
docker load < imagen.tar
```

O quizás prefiramos publicar en dockerhub (hay que hacerse una cuenta previamente. Y lo que alojemos, salvo en la versión de pago, es accesible a todo el mundo)

Ejecutamos en un terminal en el ordenador donde hemos creado la imagen:

```
docker login
docker tag elnombredenuestraimagendocker <useriddockerhub>/<repositorio>:<tag>
docker push <useriddockerhub>/<repositorio>:<tag>
```

Donde useriddockerhub es el nombre de usuario que se tenga en dockerhub, y repositorio es el nombre que estará disponible en dockerhub para la imagen. El tag es la versión

La primera línea nos hace login en dockerhub. La segunda le genera un nombre deseado para que esté en dockerhub imagen y el tercero es subir la imagen

Para finalizar, en la máquina destino nos traemos la imagen:

```
docker pull <useriddockerhub>/<repositorio>:<tag>
```

4. Crear un contenedor y lanzar la app en la máquina destino usando la imagen del paso anterior

```
docker run --rm --name nombrecontenedor -p 4500:80 nombredelaimagen
```

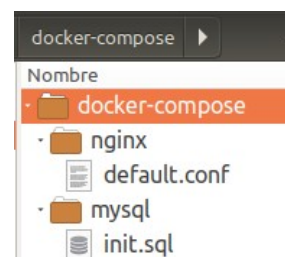
Con lo anterior le decimos que cree y lance un nuevo contenedor (run) donde el nombre del contenedor es: nombrecontenedor. La opción: --rm lo que hace es que borre el contenedor cuando se detenga la ejecución (esto se hace únicamente durante las pruebas) y le decimos en que puerto queremos que esté accesible la app. Sabemos por el Dockerfile que internamente el contenedor

sirve en el puerto 80, entonces al escribir la opción: -p 4500:80 le estamos diciendo que mapee el puerto 80 del interior del contenedor docker con el puerto de nuestra máquina 4500 Así podemos acceder a la app mediante: <http://localhost:4500>

Creando una imagen docker para laravel

En este caso estamos en una situación más compleja, porque si queremos optimizar usaremos la imagen inicial: php:fpm y adicionalmente necesitaremos un segundo contenedor docker para que haga de proxy de nuestra app con php:fpm Así que vamos a apoyarnos en una herramienta de gestión de múltiples contenedores: docker-compose

1. Hacemos una copia de nuestro proyecto laravel y dentro de la carpeta raíz creamos una nueva carpeta llamada: docker-compose que nos ayudará a localizar los ficheros que sean necesarios para luego usarlos en nuestros contenedores docker. Es habitual organizar luego esa carpeta con subcarpetas que ayuden a saber para qué servicio es el fichero. Por ejemplo si estamos en una app laravel que necesita de bases de datos tendremos la necesidad de 3 contenedores: 1º el de la app laravel que es lanzada mediante php:fpm. 2º el del servidor web nginx que es el que hace de proxy a los usuarios y se comunica con el contenedor php:fpm. 3º el del SGBD por ejemplo mysql. En la imagen se muestra una carpeta para el fichero de configuración de nginx (default.conf) y el script sql de inicio de mysql (el script que crea la base de datos, las tablas, etc)



Como estamos en un ejemplo inicial no necesitamos la carpeta de mysql. Basta con la de nginx. El fichero: raizdelproyecto/docker-compose/nginx/default.conf quedaría:

```
server {
    listen 80;
    index index.php index.html;
    server_name localhost;
    root /var/www/public;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ /\.php$ {
        fastcgi_pass app-editor-texto:9000;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        include fastcgi_params;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

Observar la etiqueta: app-editor-texto debe coincidir con lo que se ponga en docker-compose.yaml

2. Parece que con las versiones actuales hay un problema, solucionamos poniendo permisos a la carpeta raizproyecto/storage: `chmod 777 storage -R`

3. Ponemos un fichero docker-compose.yaml en la raíz del proyecto con el contenido (el ejemplo es para una app llamada: app-editor-texto) : **Nota: los ficheros yaml es importante la indentación**

```
version: '3'
services:
  app-editor-texto:
    build:
      context: .
      dockerfile: Dockerfile
    image: app-editor-texto-image
    container_name: app-editor-texto-container
    restart: unless-stopped
    tty: true

    working_dir: /var/www
    volumes:
      - ./var/www
    networks:
      - app-editor-texto-network

  app-editor-texto-webserver:
    image: nginx:stable-alpine3.20
    container_name: app-editor-texto-nginx-webserver
    restart: unless-stopped
    ports:
      - "8080:80"
    volumes:
      - ./var/www
      - ./docker-compose/nginx:/etc/nginx/conf.d/
    networks:
      - app-editor-texto-network

networks:
  app-editor-texto-network:
```

Los nombres que asignamos son libres. En esta ocasión se ha elegido poner a todos los objetos el apelativo: app-editor-texto para que quede claro que es un objeto con propósito específico para la app de editor de texto. Lo que sí que tiene que coincidir es que el nombre: app-editor-texto que hemos puesto en el services de nuestro docker-compose.yaml tiene que ser igual a lo que se ponga en el fichero nginx/default.conf de antes

Y ahora el fichero Dockerfile (lo ponemos en la raíz del proyecto)

```
# Use the official PHP image as a base image
FROM php:8.1-fpm

# Set working directory
WORKDIR /var/www

# Install system dependencies
RUN apt-get update && apt-get install -y \
    build-essential \
    libpng-dev \
    libjpeg-dev \
    libfreetype6-dev \
    locales \
    zip \
    jpegoptim optipng pngquant gifsicle \
    vim \
    unzip \
    git \
    curl \
    libzip-dev \
    libpq-dev \
    libonig-dev \
    && docker-php-ext-configure gd --with-freetype --with-jpeg \
    && docker-php-ext-install -j$(nproc) gd

# Install PHP extensions
RUN docker-php-ext-install pdo pdo_mysql pdo_pgsql mbstring zip exif pcntl

# Clear cache
RUN apt-get clean && rm -rf /var/lib/apt/lists/*

# Install Composer
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

# Copy the existing application directory contents to the working directory
COPY . /var/www

# Copy the existing application directory permissions to the working directory
COPY --chown=www-data:www-data . /var/www

RUN chown www-data:www-data . -R

# Expose port 9000 and start php-fpm server
EXPOSE 9000
CMD ["php-fpm"]
```

Para compilar nos ubicamos en la raíz del proyecto:

```
docker-compose build
```

Para lanzarla una vez compilada:

```
docker-compose up
```

Ahora vamos a explicar lo que aporta mayor información nueva. El docker-compose.yml

```
version: '3'
services:
  app-editor-texto:
    build:
      context: .
      dockerfile: Dockerfile
    image: app-editor-texto-image
    container_name: app-editor-texto-container
    restart: unless-stopped
    tty: true

    working_dir: /var/www
    volumes:
      - ./var/www
    networks:
      - app-editor-texto-network

  app-editor-texto-webserver:
    image: nginx:stable-alpine3.20
    container_name: app-editor-texto-nginx-webserver
    restart: unless-stopped
    ports:
      - "8080:80"
    volumes:
      - ./var/www
      - ./docker-compose/nginx:/etc/nginx/conf.d/
    networks:
      - app-editor-texto-network

networks:
  app-editor-texto-network:
```

Podemos observar que en docker-compose (el formato yaml trabaja con identaciones) que se ha definido dos bloques principales: services y networks. En ese mismo punto de indentación también podrían aparecer volumes

networks hace referencia a las redes distintas que nos fabricamos. Así podemos observar que hay una red privada que se llama: app-editor-texto-network y que los dos servicios (app-editor-texto y app-editor-texto-sebserver) usan esa red. Así que ambos pueden comunicarse entre sí

services hace referencia a los diferentes servicios que lanzamos (de facto los diferentes contenedores) Así está el service: app-editor-texto que vemos que se puede compilar (etiqueta build) y para el build se especifica la ubicación del Dockerfile y su nombre

tenemos la posibilidad de darle el nombre que va a tener la imagen y el contenedor. En este servicio vemos que se han puesto los nombres: app-editor-texto-image, app-editor-texto-container

restart: unless-stopped y tty: true significan que si se para el servicio se arranca automáticamente salvo que se le de la orden de parar. En cuanto a tty significa que tendremos acceso al servicio mediante un terminal.

Luego vemos que aparece una etiqueta: volumes Viene a significar que elementos se persisten (se mantienen aunque el contenedor se apague) y que mapeos se hacen con el sistema de ficheros del host anfitrión. Así la parte de:

volumes:

- ./var/www

significa que la carpeta /var/www del contenedor docker se mapea con la carpeta actual del host anfitrión. Eso va a implicar que si guardamos ficheros en storage de la app laravel van a mostrarse en el sistema de ficheros anfitrión en la ubicación actual

En el otro service: app-editor-texto-webserver vemos una línea:

ports:

- "8080:80"

que significa que el puerto interno del contenedor docker: 80 queda mapeado con el host anfitrión en el puerto 8080. Así la app está disponible en: <http://localhost:8080>

Imagen docker para laravel con persistencia en base de datos

1. Es muy parecido a lo anterior. Lo que cambia es el docker-compose.yaml y que ahora precisamos un fichero de script.sql de inicio de la base de datos. En el siguiente caso reflejamos que estuviéramos con una app que se llama: app-construcciones:

```
version: '3'
services:
  app-construcciones:
    build:
      context: .
      dockerfile: Dockerfile
    image: app-construcciones-image
    container_name: app-construcciones-container
    restart: unless-stopped
    tty: true
    environment:
      SERVICE_NAME: app
      SERVICE_TAGS: dev
    working_dir: /var/www
    volumes:
      - ./var/www
    networks:
      - app-construcciones-network

  app-construcciones-webserver:
    image: nginx:stable-alpine3.20
    container_name: app-construcciones-nginx-webserver
    restart: unless-stopped
    ports:
      - "8080:80"
    volumes:
      - ./var/www
      - ./docker-compose/nginx:/etc/nginx/conf.d/
    networks:
      - app-construcciones-network

  app-construcciones-mysqldb:
    image: mysql:8.3.0
    container_name: app-construcciones-mysql-container
    environment:
      MYSQL_ROOT_PASSWORD: 1q2w3e4r
    volumes:
      - app-construcciones-dbdata:/var/lib/mysql
      - ./docker-compose/init.sql:/docker-entrypoint-initdb.d/init.sql
    networks:
      - app-construcciones-network

volumes:
  app-construcciones-dbdata:

networks:
  app-construcciones-network:
```

Las diferencias con el anterior están en el contenedor mysql. Vemos que se le da la password de root desde environment. Y establecemos dos volúmenes para la persistencia:

volumes:

- app-construcciones-database:/var/lib/mysql
- ./docker-compose/init.sql:/docker-entrypoint-initdb.d/init.sql

Con lo anterior se está diciendo que en el volumen: app-construcciones-database se va a almacenar la información del contenedor en concreto de la carpeta /var/lib/mysql que es la que guarda la información de las bases de datos. Observar que NO sabemos en qué punto del sistema de ficheros del anfitrión se está guardando esa información. Habría que leer mediante: docker volumes y a partir de ahí obtener la información

En cuanto a la segunda ruta es la forma que se hace habitualmente para pasarle un script sql de inicio a la base de datos. El fichero que tenemos ubicado en: docker-compose/init.sql se copia en el contenedor docker en el punto que él espera que esté: /docker-entrypoint-initdb.d/init.sql

2. Como en el caso anterior tenemos que modificar el fichero nginx/default.conf para que aparezca el nombre del contenedor de nuestra app:

```
server {
    listen 80;
    index index.php index.html;
    server_name localhost;
    root /var/www/public;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ /\.php$ {
        fastcgi_pass app-construcciones:9000;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        include fastcgi_params;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

3. Como antes también hay que dar permisos a storage: `chmod 777 storage`

4. Hay que modificar el fichero **.env** en la raíz del proyecto para que apunte en DB_HOST al nombre del servicio de base de datos:

```
DB_CONNECTION=mysql
DB_HOST=app-construcciones-mysql-container
DB_PORT=3306
DB_DATABASE=construcciones
DB_USERNAME=root
DB_PASSWORD=1q2w3e4r
```

El resto es el mismo procedimiento que antes